

SEGURIDAD EN BASE DE DATOS



**Universidad
Europea**

ACTIVIDAD 1:

Desplegando PostgreSQL con Docker y Configuración básica

Estudiante: Iris Carolina Fernández González

Profesor: Miguel Angel Couto Caro

Grado: Ingeniería de la Ciberseguridad

Curso Académico: 2025-2026

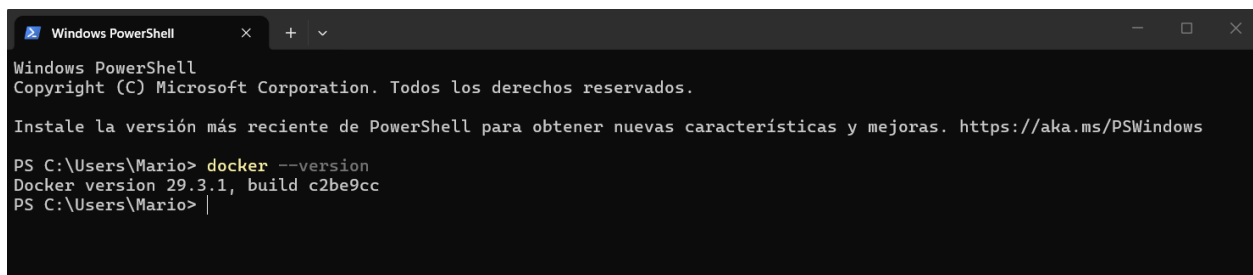
1- Configurar el entorno Docker

Para comenzar la práctica, hice la instalación de Docker Desktop en mi sistema Windows.

Durante la instalación habilite el uso de WSL2, lo que permite ejecutar contenedores basados en Linux dentro de Windows.

Una vez completada la instalación, he verificado su correcto funcionamiento mediante el siguiente comando: `docker --version`

El resultado muestra que Docker se encuentra correctamente instalado y en ejecución.

A screenshot of a Windows PowerShell terminal window. The title bar says 'Windows PowerShell'. The text inside shows the copyright notice for Microsoft Corporation, a link to update PowerShell, and the command 'docker --version' being executed. The output is 'Docker version 29.3.1, build c2be9cc'.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

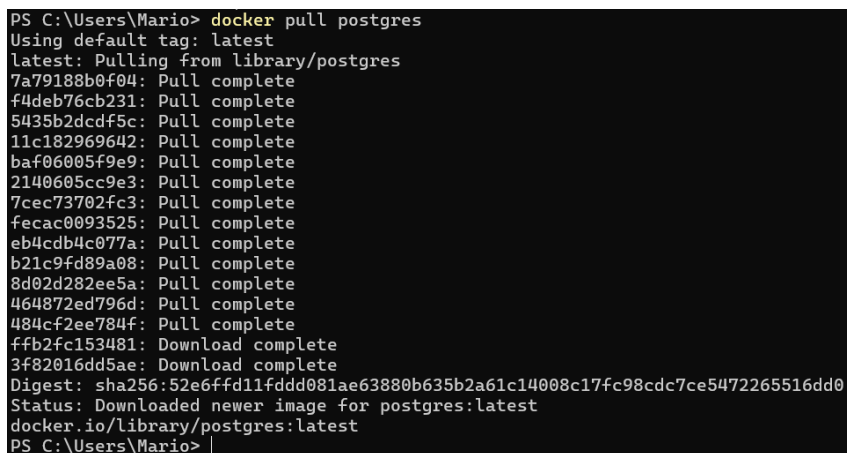
Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\Mario> docker --version
Docker version 29.3.1, build c2be9cc
PS C:\Users\Mario> |
```

2- Desplegar PostgreSQL Usando Docker

A continuación, he descargado la imagen oficial de PostgreSQL desde Docker Hub utilizando el siguiente comando: `docker pull postgres`

Esta imagen contiene todo lo necesario para ejecutar un servidor de base de datos PostgreSQL dentro de un contenedor Docker.

A screenshot of a Windows PowerShell terminal window showing the command 'docker pull postgres' and its output. The output shows the image being pulled from the Docker library, with a list of layers being pulled and their status (Pull complete). It also shows the digest and status of the download.

```
PS C:\Users\Mario> docker pull postgres
Using default tag: latest
latest: Pulling from library/postgres
7a79188b0f04: Pull complete
f4deb76cb231: Pull complete
5435b2dcd5c: Pull complete
11c182969642: Pull complete
baf06005f9e9: Pull complete
2140605cc9e3: Pull complete
7cec73702fc3: Pull complete
fecac0093525: Pull complete
eb4cdb4c077a: Pull complete
b21c9fd89a08: Pull complete
8d02d282ee5a: Pull complete
46d872ed796d: Pull complete
484cf2ee784f: Pull complete
ffb2fc153481: Download complete
3f82016dd5ae: Download complete
Digest: sha256:52e6ffd11fddd081ae63880b635b2a61c14008c17fc98cdc7ce5472265516dd0
Status: Downloaded newer image for postgres:latest
docker.io/library/postgres:latest
PS C:\Users\Mario> |
```

Una vez descargada la imagen, he creado y ejecutado un contenedor de PostgreSQL utilizando el siguiente comando: `docker run --name mi_postgres -e POSTGRES_PASSWORD=1234 -d -p 5432:5432 postgres`

Este comando crea un contenedor llamado "mi_postgres", establece una contraseña para el usuario administrador y expone el servicio en el puerto 5432.

Posteriormente, he verificado que el contenedor se encontraba en ejecución mediante el comando: `docker ps`

Tal y como se observa en la captura, el contenedor está activo y funcionando correctamente.

```
PS C:\Users\Mario> docker run --name mi_postgres -e POSTGRES_PASSWORD=1234 -d -p 5432:5432 postgres
4790059d1257841fe229b4f9352349481c1d60160412a77ef8728996bd932852
PS C:\Users\Mario> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                                                                 NAMES
4790059d1257   postgres "docker-entrypoint.s..." 22 seconds ago Up 22 seconds  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp  mi_postgres
PS C:\Users\Mario> |
```

3- Acceder al contenedor de PostgreSQL

Una vez el contenedor se encontraba en ejecución, he accedido a la base de datos PostgreSQL mediante el siguiente comando: `docker exec -it mi_postgres psql -U postgres`

Esto permitió abrir una consola interactiva de PostgreSQL, desde la cual se pueden ejecutar consultas SQL directamente sobre la base de datos.

```
PS C:\Users\Mario> docker exec -it mi_postgres psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

postgres=# |
```

4- Configuración básica

Una vez dentro de PostgreSQL, he realizado la configuración básica.

En primer lugar, he creado una nueva base de datos llamada "mi_base":

```
CREATE DATABASE mi_base;
```

Luego he creado un usuario con contraseña:

```
CREATE USER usuario1 WITH PASSWORD '1234';
```

Finalmente, se otorgaron todos los privilegios sobre la base de datos al usuario creado:

```
GRANT ALL PRIVILEGES ON DATABASE mi_base TO usuario1;
```

Estas acciones permiten establecer un control inicial sobre el acceso a la base de datos.

```
postgres=# CREATE DATABASE mi_base;
CREATE DATABASE
postgres=# CREATE USER usuario1 WITH PASSWORD '1234';
CREATE ROLE
postgres=# GRANT ALL PRIVILEGES ON DATABASE mi_base TO usuario1;
GRANT
postgres=# |
```

5- Definir roles y niveles de acceso

Crear Roles:

En esta parte de la práctica he definido diferentes roles para controlar el acceso a los datos.

Se han creado 2 roles:

- rol_lectura: con permisos únicamente de lectura.
- rol_lectura_escritura: con permisos de lectura, inserción, actualización y eliminación.

```
postgres=# CREATE ROLE rol_lectura;
CREATE ROLE
postgres=# CREATE ROLE rol_lectura_escritura;
CREATE ROLE
```

Crear Tablas y asignar permisos:

Luego he creado las tablas employees y salaries en PostgreSQL. La tabla salaries incluye una clave foránea que referencia a employees, estableciendo una relación entre empleados y sus salarios.

También he asignado permisos específicos a los roles creados. El rol de solo lectura únicamente puede consultar información de las tablas, mientras que el rol de lectura y escritura tiene permisos completos para consultar, insertar, actualizar y eliminar registros.

```
postgres=# CREATE TABLE employees (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  position VARCHAR(100)
);

CREATE TABLE salaries (
  id SERIAL PRIMARY KEY,
  employee_id INT REFERENCES employees(id),
  amount DECIMAL
);

CREATE TABLE
CREATE TABLE
postgres=# GRANT SELECT ON employees TO rol_lectura;
GRANT SELECT ON salaries TO rol_lectura;
GRANT
GRANT
postgres=# GRANT SELECT, INSERT, UPDATE, DELETE ON employees TO rol_lectura_escritura;
GRANT SELECT, INSERT, UPDATE, DELETE ON salaries TO rol_lectura_escritura;
GRANT
GRANT
postgres=# |
```

Crear usuarios y asignarles roles:

He creado 2 usuarios en la base de datos PostgreSQL: uno con acceso de solo lectura y otro con acceso de lectura y escritura. A cada usuario se le asignó un rol previamente definido para controlar sus permisos sobre las tablas.

```
postgres=# CREATE USER user_read WITH PASSWORD '1234';
GRANT rol_lectura TO user_read;
CREATE ROLE
GRANT ROLE

postgres=# CREATE USER user_write WITH PASSWORD '1234';
GRANT rol_lectura_escritura TO user_write;
CREATE ROLE
GRANT ROLE
postgres=# |
```

6- Configuración de Portainer (opcional)

En esta parte he instalado y configurado Portainer como herramienta de administración gráfica para Docker. Portainer permite gestionar contenedores, imágenes y volúmenes a través de una interfaz web, facilitando la administración del entorno.

He realizado la instalación de Portainer mediante Docker utilizando el comando `docker run`. Posteriormente he verificado la creación y ejecución del contenedor mediante `docker ps`, confirmando que el servicio estaba activo.

He accedido a la interfaz web de Portainer mediante la dirección <http://localhost:9000>, donde se configuró un usuario administrador para el acceso al sistema.

Posteriormente, he seleccionado el entorno Docker local y se verificó la ejecución de los contenedores desplegados, incluyendo el contenedor de PostgreSQL utilizado en la práctica.

Gracias a Portainer, fue posible visualizar de forma clara el estado de los contenedores, confirmando que el servicio de base de datos se encontraba en ejecución correcta.

```
PS C:\Users\Mario> docker run -d -p 9000:9000 -p 8000:8000 --name portainer --restart=always -v /var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer-ce
Unable to find image 'portainer/portainer-ce:latest' locally
latest: Pulling from portainer/portainer-ce
4f4fb700ef54: Pull complete
c7e34731674e: Pull complete
224552a17e45: Pull complete
2ff62971c2fb: Pull complete
1c4fea3af7a0: Pull complete
d1e89ff9b379: Pull complete
70636407b8c2: Pull complete
9ab41045230a: Pull complete
02a2cded7f75: Download complete
662b373d6a7d: Download complete
Digest: sha256:1ae8e65d50ca5498cb2c33e617495a1e3ef245b0d2392b4a44c70ae09b822891
Status: Downloaded newer image for portainer/portainer-ce:latest
6f2068efccbeccfebc3247f26a50688a7d24cedc07aee53fa4ad507979ab8bed
PS C:\Users\Mario> |
```

```
PS C:\Users\Mario> docker ps
```

CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED	STATUS	PORTS
6f2068efccbe	portainer/portainer-ce	"/portainer"	portainer	14 minutes ago	Up 5 minutes	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
4790059d1257	postgres	"docker-entrypoint.s..."	mi_postgres	About an hour ago	Up About an hour	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

```
PS C:\Users\Mario> |
```

localhost:9000/#/init/admin

Photographer ion2x - Twitch

Todos los marcadores

PORTAINER.IO

New Portainer installation

Please create the initial administrator user.

Username

admin

Password

Confirm password

x

The password must be at least 12 characters long.

Create user

Restore Portainer from backup

← → ↻

localhost:9000/#/wizard

🔑 ☆ 📄 ⬇️ 🌙 ⋮

🔑 Photographer 📺 ion2x - Twitch

Todos los marcadores

Upgrade to Business Edition

PORTAINER.IO
COMMUNITY EDITION

🏠 Home

Environment: / None selected

Administration

- User-related
- Environment-related
- Registries
- Logs
- Notifications
- Settings

Portainer Community Edition 2.39.1 LTS

Environment Wizard

Quick Setup

🔧 Environment Wizard

Welcome to Portainer

We have connected your local environment of Docker to Portainer.
Get started below with your local portainer or connect more container environments.



Get Started

Proceed using the local environment which Portainer is running in



Add Environments

Connect to other environments

← → ↻

localhost:9000/#/home

🔑 ☆ 📄 ⬇️ 🌙 ⋮

🔑 Photographer 📺 ion2x - Twitch

Todos los marcadores

Upgrade to Business Edition

PORTAINER.IO
COMMUNITY EDITION

🏠 Home

local

Dashboard

Templates

Stacks

Containers

Images

Networks

Volumes

Events

Host

Administration

- User-related
- Environment-related

Portainer Community Edition 2.39.1 LTS

Environments

Home

Latest News From Portainer

Docker v29.3.0 fixes a critical CVE but also has breaking changes, especially for Swarm users. Learn more about [the CVE](#) and the [breaking changes](#) in our blogs.

Environments

Search by name, group, tag, status, URL... ✕

Refresh

Kubeconfig

Click on an environment to manage

Platform Connectio... Status Tags Groups Agent Vers... Clear all Sort By



local

🟢 Up

2026-04-12 13:39:56 Standalone 29.3.1 /var/run/docker.sock

Group: Unassigned No tags Local

0 stacks 2 containers 2 0 0 0 2 volumes 2 images 20 CPU 8.2 GB RAM

Disconnect

Connected

Items per page 10

localhost:9000/#/3/docker/containers

Photographer ion2x - Twitch

Upgrade to Business Edition

PORTAINER.IO COMMUNITY EDITION

Home

local

Dashboard

Templates

Stacks

Containers

Images

Networks

Volumes

Events

Host

Administration

User-related

Environment-related

Containers

Container list

Containers

Search...

Start Stop Kill Restart Pause Resume Remove Add container

Name	State	Quick Actions	Stack	Image	Created	IP Address	Published Ports	Ownership
ml_postgres	running	Logs Inspect Stats Console Attach	-	postgres	2026-04-12 12:21:44	172.17.0.2	5432:5432	administrators
portainer	running	Logs Inspect Stats Console Attach	-	portainer/portainer-ce	2026-04-12 13:30:33	172.17.0.3	8000:8000 9000:9000	administrators

Items per page 10

localhost:9000/#/3/docker/containers/4790059d1257841fe229b4f9352349481c1d60160412a77ef8728996bd932852

Photographer ion2x - Twitch

Upgrade to Business Edition

PORTAINER.IO COMMUNITY EDITION

Home

local

Dashboard

Templates

Stacks

Containers

Images

Networks

Volumes

Events

Host

Administration

User-related

Environment-related

Containers > ml_postgres

Container details

Actions

Start Stop Kill Restart Pause Resume Remove Recreate Duplicate/Edit

Container status

ID	4790059d1257841fe229b4f9352349481c1d60160412a77ef8728996bd932852
Name	ml_postgres
Status	Running for about 1 hour
Created	2026-04-12 12:21:44
Start time	2026-04-12 12:21:44
Container webhook	☐ Business Feature
Logs Inspect Stats Console Attach	

Cómo ejecutar los archivos entregados:

1. Ejecución del archivo Bash (.sh)

El archivo Bash contiene los comandos necesarios para automatizar la configuración inicial del entorno Docker y el despliegue de PostgreSQL.

Para ejecutarlo, se deben seguir los siguientes pasos:

1. Abrir una terminal (PowerShell en Windows o terminal en Linux/Mac).
2. Ubicarse en el directorio donde se encuentra el archivo .sh.
3. Dar permisos de ejecución al archivo (en caso de Linux/Mac): `chmod +x nombre_del_archivo.sh`
4. Ejecutar el script: `./nombre_del_archivo.sh`

En el caso de Windows, puede ejecutarse utilizando Git Bash o ejecutando los comandos manualmente en PowerShell.

Este script realiza:

- La verificación de Docker
- La descarga de la imagen de PostgreSQL
- La creación y ejecución del contenedor
- El acceso a la base de datos PostgreSQL dentro del contenedor

2. Ejecución del archivo SQL (.sql)

El archivo SQL contiene toda la configuración de la base de datos, incluyendo la creación de tablas, roles, usuarios y asignación de permisos.

Para ejecutarlo:

1. Acceder al contenedor de PostgreSQL: `docker exec -it mi_postgres psql -U postgres`
2. Crear la base de datos (si no existe): `CREATE DATABASE nombre_basedatos;`
3. Conectarse a la base de datos: `\c nombre_basedatos`
4. Ejecutar el archivo SQL: `\i ruta_del_archivo.sql`

Este script configurará automáticamente:

- Las tablas de la base de datos
- Los roles de acceso (lectura y escritura)
- Los usuarios del sistema
- Los permisos correspondientes